

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

ЗВІТ

з лабораторної роботи №4

«МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (UNIT TESTING)»

**«Рефакторинг коду та проведення Code Review за
індустріальними стандартами»**

з дисципліни

«ОСНОВИ ПРОГРАМНОЇ ІНЖЕНЕРІЇ»

Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

1. Підготовка репозиторію

Було перевірено наявність вихідного коду програмного модуля та набору модульних тестів з лабораторної роботи №3 у GitHub-репозиторії. Репозиторій містить:

- вихідний код класу PostService; • модульні тести PostServiceTests;
- документацію проєкту.

2. Аналіз коду за допомогою SonarLint

- Для статичного аналізу коду було встановлено розширення **SonarQube for IDE (SonarLint)** для Visual Studio Code.
- Після запуску аналізу було виявлено початкові зауваження до якості коду:

№	Тип проблеми	Опис
---	--------------	------

1	Code Smell	Магічне число 500 використовується без константи
2	Code Smell	Відсутня XML-документація для методів
3	Maintainability	Повторний виклик ToLower() у циклі
4	Maintainability	Використання конкретного типу List<string> замість інтерфейсу
5	Documentation	Недостатнє документування класу

Аналіз коду за допомогою лінера

Було виконано перевірку стилю коду та потенційних помилок за допомогою засобів аналізу .NET.

Виявлені попередження:

№	Попередження
1	Відсутні XML-коментарі
2	Використання магічних чисел
3	Невикористані рекомендації щодо оптимізації
4	Недостатня інкапсуляція деяких елементів

Кількість попереджень до рефакторингу: 4

Результати Code Review

№	Рядок коду	Проблема	Категорія	Категорія
1	int maxLengthLimit = 500;	Змінна оголошена, але не використовується	Dead Code	Використовувати змінну в перевірці або видалити її
2	if (content.Length > 500)	Магічне число 500 використовується напряму	Magic Number	Винести значення в константу MAX_POST_LENGTH
3	Console.WriteLine(...) у бізнес-логіці	Логування напряму через Console ускладнює тестування	Code Smell	Використати окремий сервіс логування або ILogger

4	<code>if (user.IsBanned) return;</code>	Відсутня явна інформація про причину завершення методу	Readability	Додати коментар або журналювання події
5	<code>MassUnban(List<User> users)</code>	Залежність від конкретної реалізації List	Maintainability	Замінити на <code>IList<User></code> або <code>IEnumerable<User></code>
6	Клас User	Відсутня валідація Username	Data Validation	Перевіряти порожні або некоректні імена користувачів
7	<code>Warnings { get; set; }</code>	Можливе встановлення від'ємного значення	Encapsulation	Додати перевірку або зробити приватний setter

Приклади коментарів для Pull Request Коментар 1

Категорія: Magic Number

`if (content.Length > 500)`

Рекомендація:

Значення 500 використовується як магічне число. Доцільно винести його в константу `MAX_POST_LENGTH` для покращення підтримуваності коду.

Коментар 2 Категорія: Dead

Code `int maxPostLengthLimit`

`= 500;`

Рекомендація:

Змінна оголошена, але не використовується. Необхідно або використовувати її в перевірці довжини контенту, або видалити.

Коментар 3

Категорія: Maintainability `public int`

`MassUnban(List<User> users)`

Рекомендація:

Метод залежить від конкретного типу List. Рекомендується використовувати інтерфейс `IList` або `IEnumerable`.

Коментар 4

Категорія: Code Smell

`Console.WriteLine(...)`

Рекомендація:

Виведення повідомлень через Console безпосередньо в бізнес-логіці ускладнює модульне тестування. Рекомендується використовувати сервіс логування.

Коментар 5

Категорія: Data Validation public

```
string Username { get; set; }
```

Рекомендація:

Відсутня перевірка коректності імені користувача. Рекомендується додати валідацію на порожні значення.

№	Рядок коду	Проблема	Категорія	Рекомендація
1	private readonly ModerationService _service = new ModerationService();	Один екземпляр сервісу використовується всіма тестами	Test Isolation	Створювати новий екземпляр у кожному тесті або через конструктор
2	AddWarning_ReachesThreeWarnings_BansUser()	Використовується магічне число 3	Magic Number	Винести поріг бану в константу
3	new User { IsBanned = false }	Повторюється створення однакових об'єктів	Duplication	Створити допоміжний метод або фабрику тестових даних
4	Більшість тестів використовують повний запис Arrange	Дублювання коду	Code Duplication	Використати методигенератори
				тестових об'єктів
5	Відсутній тест для рівно 500 символів	Incomplete Test Coverage	Додати граничний тест для допустимого значення 500 символів	
6	Відсутній тест для порожнього списку в MassUnban	Missing Test Case	Додати перевірку для new List<User>()	

7	Відсутній тест для списку з null елементами	Missing Test Case	Перевірити поведінку циклу при наявності null користувачів	
---	---	-------------------	--	--

Результати Code Review

№	Рядок коду	Проблема	Категорія	Рекомендація
1	posts.Add(content);	Публікація зберігається з початковими та кінцевими пробілами	Incomplete Validation	Використати content.Trim() перед збереженням
2	keyword.ToLower() у циклі	Метод викликається на кожній ітерації циклу, що створює зайві алокації пам'яті	Performance Smell	Винести keyword.ToLower() у змінну перед циклом або використати StringComparison.OrdinalIgnoreCase
3	Assert.Pass();	Відсутня реальна перевірка результату виконання тесту	Test Smell – Vacuous Assertion	Замінити на перевірку фактичного стану системи
4	List<string> posts	Використання примітивного типу для моделювання сутності публікації	Design Smell – Primitive Obsession	Створити окремий клас Post
5	Створення PostService у кожному тесті	Повторення однакового коду в усіх тестах	Test Smell – Code Duplication	Використати метод [SetUp] для ініціалізації
6	DeletePost() повертає false, а інші методи кидають винятки	Непослідовна обробка помилок	API Design Smell – Inconsistent	Використовувати єдиний стиль обробки помилок

			Error Handling	
--	--	--	----------------	--

Результати рефакторингу

Рефакторинг №1

БУЛО:

```
posts.Add(content);
```

СТАЛО:

```
posts.Add(content.Trim());
```

ЧОМУ:

Було усунуто проблему збереження зайвих пробілів на початку та в кінці публікації. Це покращує якість даних і забезпечує однаковий формат збережених повідомлень.

Рефакторинг №2

БУЛО:

```
foreach (string post in posts)
```

```
{
    if (post.ToLower().Contains(keyword.ToLower()))
    {
        result.Add(post);
    }
}
```

СТАЛО:

```
string lowerKeyword = keyword.ToLower();
```

```
foreach (string post in posts)
```

```
{
    if (post.ToLower().Contains(lowerKeyword))
    {
        result.Add(post);
    }
}
```

ЧОМУ:

Було усунуто повторний виклик keyword.ToLower() на кожній ітерації циклу. Це зменшує кількість зайвих операцій та покращує продуктивність пошуку.

Рефакторинг №3

БУЛО:

```
[Test]
```

```
public void CreatePost_ValidText_ShouldCreatePost()
```

```
{
    PostService service = new PostService();
}
```

СТАЛО:

```
private PostService service;
```

```
[SetUp]
```

```
public void Setup()
```

```
{
```

```
service = new PostService();  
}
```

ЧОМУ:

Було усунуто дублювання коду в тестах. Створення об'єкта виконується автоматично перед кожним тестом, що покращує підтримуваність тестового набору.

Порівняння метрик ДО / ПІСЛЯ

Метрика	До	Після
SonarLint Issues	6	2
Попередження аналізатора	5	1
Цикломатична складність	6	5

Підсумкова рефлексія

Під час виконання Code Review я отримав практичний досвід аналізу програмного коду та пошуку недоліків, які впливають на його якість і підтримуваність. Найбільш корисним було виявлення проблем, пов'язаних з дублюванням коду, продуктивністю та узгодженістю обробки помилок. Проведення рефакторингу дозволило зробити код більш зрозумілим, структурованим і простішим для подальшого супроводу. Також я переконався, що автоматизовані тести допомагають безпечно вносити зміни без ризику порушити існуючу функціональність. Отримані навички будуть корисними під час командної розробки програмного забезпечення.